Customer Support Ticket Management System
Extended Report

The original project implemented a modular, C-based customer support ticket management system that utilized both linear (queues) and non-linear (graphs) data structures. The functionalities to be implemented were: Creation of ticket, Assignment of agents, Linking related tickets, Priority escalation. Added functionalities are Searching by keyword and SLA report generation.

Members:
AL Sivagami - PES1UG24CS048
Advitha Manish - PES1UG24CS035
Anagha B Poojari - PES1UG24CS056

The functionalities code and output:

1. Creating a ticket

```c
Ticket* createTicket(int id, char *customerName, char *issue, int priority) {
    Ticket *t = (Ticket*) malloc(sizeof(Ticket));

    if (!t) {
        printf("Memory allocation failed for ticket.\n");
        exit(1);
    }

    t->id = id;
    strcpy(t->customerName, customerName);
    strcpy(t->issue, issue);
    t->priority = priority;
    t->assignedAgentId = -1; // unassigned initially
    t->isResolved = 0;       // all are open initially
    t->createdAt = time(NULL);

    printf("Ticket #%d created successfully!\n", t->id);
    return t;
}
```

```
========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 1
Enter customer name: Akshya
Enter issue: Payment issue
Enter priority (1=High, 2=Medium, 3=Low): 2
Ticket #1 created successfully!
Ticket #1 added to queue successfully.
```

2. Assignment of Agents

```c
void assignAgent(Ticket *t, Agent *a) {
    if (!a->isAvailable) {
        printf("Agent %s is currently busy.\n", a->name);
        return;
    }

    t->assignedAgentId = a->id;
    a->isAvailable = 0;
    printf("Agent %s assigned to Ticket #%d successfully.\n", a->name, t->id);
}
```

```
========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 2
Enter Ticket ID: 2
Enter Agent ID (101-Rahul, 102-Meera, 103-Arjun): 101
Agent Rahul assigned to Ticket #2 successfully.
```

3. Linking related Tickets

```c
void addLink(Graph *g, int t1, int t2) {
    // Adjusting because our ticket IDs start from 1 (not 0)
    t1--;
    t2--;

    if (t1 < 0 || t1 >= g->numTickets || t2 < 0 || t2 >= g->numTickets) {
        printf("Invalid ticket IDs.\n");
        return;
    }

    g->adjMatrix[t1][t2] = 1;
    g->adjMatrix[t2][t1] = 1; // undirected
    printf("Ticket #%d and Ticket #%d linked successfully.\n", t1 + 1, t2 + 1);
}

void showRelatedTickets(Graph *g, int ticketId) {
    ticketId--;

    if (ticketId < 0 || ticketId >= g->numTickets) {
        printf("Invalid Ticket ID.\n");
        return;
    }

    printf("\nTickets related to #%d: ", ticketId + 1);
    int hasRelated = 0;

    for (int j = 0; j < g->numTickets; j++) {
        if (g->adjMatrix[ticketId][j] == 1) {
            printf("%d ", j + 1);  // print as 1-based ID
            hasRelated = 1;
        }
    }

    if (!hasRelated)
        printf("None");
    printf("\n");
    return;
}
```

```
========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 3
Enter Ticket IDs to link (e.g. 1 2): 1 4
Ticket #1 and Ticket #4 linked successfully.

========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 7
Enter Ticket ID to view related tickets: 1

Tickets related to #1: 4
```

4. Priority Escalation

```c
void escalatePriority(TicketQueue *q, int ticketId) {
    if (isQueueEmpty(q)) {
        printf("No tickets in the queue.\n");
        return;
    }

    for (int i = q->front; i <= q->rear; i++) {
        if (q->tickets[i]->id == ticketId) {
            if (q->tickets[i]->priority == 1) {
                printf("Ticket #%d is already at highest priority.\n", ticketId);
                return;
            }
            q->tickets[i]->priority--;
            printf("Ticket #%d priority escalated successfully.\n", ticketId);
            return;
        }
    }

    printf("Ticket with ID %d not found.\n", ticketId);
}
```

```
========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 4
Enter Ticket ID to escalate: 4
Ticket #4 priority escalated successfully.

========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 5
Enter Ticket ID to view details: 4

--- Ticket Details ---
Ticket ID: 4
Customer Name: Anushka
Issue: Transaction failed
Priority: 1
Agent ID: -
Status: Open
```

5. Search by Keyword

To enable users to search tickets based on words or phrases present in either the customer name, or the issue description.

```c
// Search tickets by keyword in customer name or issue description
void searchTicketsByKeyword(TicketQueue *q, const char *keyword) {
    if (isQueueEmpty(q)) {
        printf("No tickets available.\n");
        return;
    }

    char lowerKeyword[100];
    strcpy(lowerKeyword, keyword);
    toLowerCase(lowerKeyword);

    int found = 0;
    printf("\n=========== SEARCH RESULTS ===========\n");

    for (int i = q->front; i <= q->rear; i++) {
        Ticket *t = q->tickets[i];
        char nameCopy[MAX_NAME_LEN];
        char issueCopy[MAX_ISSUE_LEN];

        strcpy(nameCopy, t->customerName);
        strcpy(issueCopy, t->issue);

        toLowerCase(nameCopy);
        toLowerCase(issueCopy);

        if (strstr(nameCopy, lowerKeyword) || strstr(issueCopy, lowerKeyword)) {
            printf("\nTicket ID: %d\n", t->id);
            printf("Customer: %s\n", t->customerName);
            printf("Issue: %s\n", t->issue);
            printf("Priority: %d\n", t->priority);
            printf("Resolved: %s\n", t->isResolved ? "Yes" : "No");
            found = 1;
        }
    }
```

```
========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 11
Enter keyword to search: payment

=========== SEARCH RESULTS ===========

Ticket ID: 1
Customer: Akshya
Issue: Payment issue
Priority: 2
Resolved: No
======================================
```

6. SLA Compliance Report

To track resolution performance based on priority based time limits (Service Level Agreements).

The time taken (in hours) is compared with the SLA limit:

High Priority → 4 hrs

Medium → 24 hrs

Low → 72 hrs

```c
// Print all resolved tickets with SLA status
void generateSLAReport(TicketQueue *q) {
    if (isQueueEmpty(q)) {
        printf("No tickets available.\n");
        return;
    }

    printf("\n=========== SLA COMPLIANCE REPORT ===========\n");
    printf("ID | Priority | Status      | Time Taken (hrs)\n");
    printf("---------------------------------------------\n");

    for (int i = q->front; i <= q->rear; i++) {
        Ticket *t = q->tickets[i];
        if (t->isResolved) {
            double timeTaken = difftime(t->resolvedAt, t->createdAt) / 3600.0;
            int slaLimit;
            if (t->priority == 1) slaLimit = SLA_HIGH;
            else if (t->priority == 2) slaLimit = SLA_MEDIUM;
            else slaLimit = SLA_LOW;

            const char *status = (timeTaken <= slaLimit) ? "Met" : "Breached";
            printf("%2d | %-8d | %-10s | %.2f\n",
                   t->id, t->priority, status, timeTaken);
        }
    }
    printf("---------------------------------------------\n");
}
```

```
========= CUSTOMER SUPPORT TICKETER =========
1. Create Ticket
2. Assign Agent
3. Link Related Tickets
4. Escalate Priority
5. Display Ticket by ID
6. Display All Tickets
7. Show Related Tickets
8. Resolve Ticket
9. Exit
10. Generate SLA report
11. Search Tickets by Keyword
Enter your choice: 10

=========== SLA COMPLIANCE REPORT ===========
ID | Priority | Status      | Time Taken (hrs)
---------------------------------------------
 1 | 2        | Met         | 0.52
 2 | 2        | Met         | 0.01
---------------------------------------------
```